

An enterprise agent should not be trusted. It should be *compiled*.

Lattice is a governed context plane between a question and an action. It turns natural language into a signed, verifiable execution plan — or into a refusal it can defend.

4

RUNTIME OUTCOMES

9

INDUSTRY ONTOLOGIES

34

DESCRIBED API ROUTES

8

CONNECTOR PROVIDERS

Ed25519 / ES256

PLAN SIGNING

Every agent program stalls in the same place, and it is never the model.

Pilots work. Production doesn't. The blocker shows up the first time someone from risk, legal, or the data governance team asks four questions about a single agent action — after it has already happened.

Q1

What did it think the question meant?

Intent resolution happens inside a prompt. There is no record of the candidates considered or why one won.

Q2

Which system was it allowed to read?

The agent runs as a service principal with far broader grants than the person who asked.

Q3

Under what policy, on whose authority?

Approval, if it happened, lives in a Slack thread. Purpose was never a field.

Q4

What would have made it stop?

Nothing. The system had no way to say *I cannot support this answer.*

→ A chat transcript is not an answer to any of the four. Neither is a log file.

Four ways a working demo becomes an incident.

These are not edge cases. They are the default behaviour of an ungoverned agent wired to real systems.

01 – Semantic drift

“Exposure” in the prompt is not “exposure” in the warehouse.

Two definitions coexist. They disagree by a material amount. Nobody notices until the number is in front of a regulator, and finance sides with the warehouse.

02 – Decorative permissions

The caller declares its own entitlements.

If a request body can assert what it is allowed to do, every permission gate in the system is a tautology. Row filters and column masks in Unity Catalog or Fabric never see the person asking.

03 – No floor on confidence

Asked something it cannot support, it answers anyway.

Abstention was never an available output. A weak or stale answer is indistinguishable, in shape and in tone, from a well-evidenced one.

04 – Unreconstructable audit

Explaining the action means re-reading prompts.

There is no artifact that pins what the system meant, what it read, which policy version applied, and who authorised it. Six months later, nobody can produce one.

→ Each of these is a governance failure, not an accuracy failure. Better models do not fix any of them.

A question does not return an answer. It returns a *decision*.

Lattice compiles a natural-language question against a published **Context Contract** — a versioned package of meaning, bindings, evidence, policy, and permitted operations for one bounded workflow.

Compilation is deterministic and produces exactly one of four outcomes. Three of them are refusals, and each refusal carries a reason code a governance team can act on.

Separation of concern

The compiler is pure. It cannot execute.

It produces an unsigned plan or a non-execution decision, and nothing else. It holds no key, opens no connection, and calls no source system.

Separation of trust

The API is the only thing that signs.

It verifies identity against your issuer, signs the plan, and pins the contract release into it. Execution is a separate, separately authorised call.

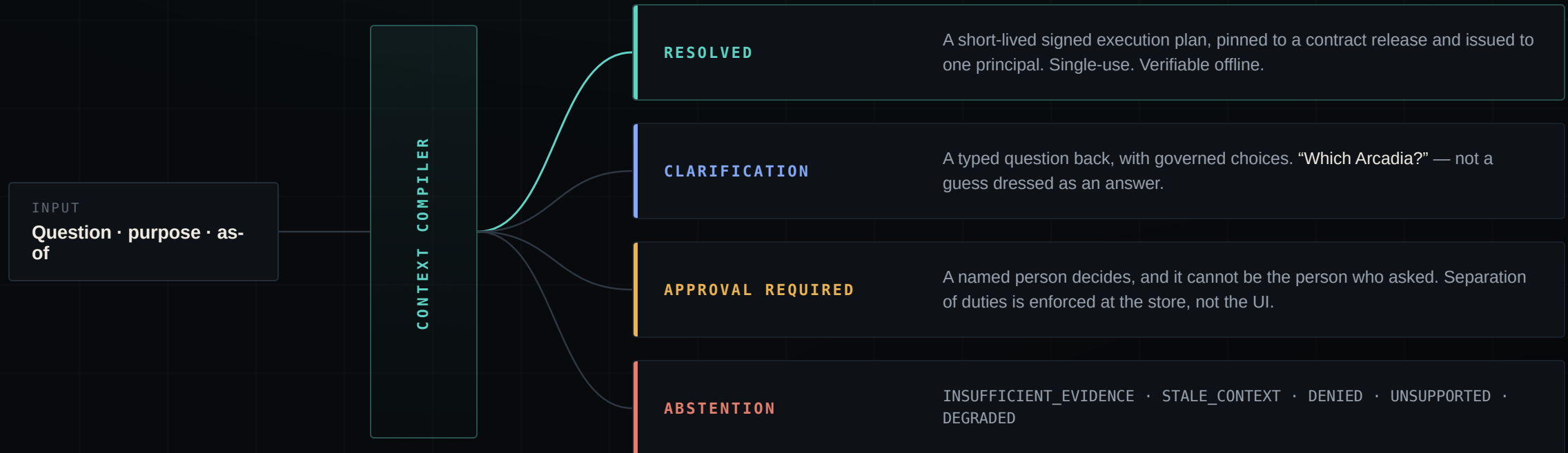
Separation of authority

The executor verifies without asking us.

@lattice/plan-verifier checks signature, expiry, subject and release pin from a published JWKS. No call back to Lattice.

Clarification and abstention are *successes*.

Most systems have one output and a failure state. Lattice has four outputs, and the governance team wants three of them.



→ Four risk tiers set the thresholds. An INFORMATIONAL lookup and an OPERATIONAL_ACTION do not get the same benefit of the doubt.

The artifact you ship is not an ontology. It is a *contract*.

A graph is a picture of what you mean. A Context Contract is a versioned, digest-addressed package that a runtime can enforce — and that a reviewer can approve, diff, roll back, and retire.

01

Competency questions

The decisions this contract exists to answer, each with an expected answer shape, an owner, and an impact level.

02

Concept scope

A version-pinned selection from the workspace ontology. The contract inherits meaning; it does not fork it.

03

Source bindings

Which system answers which property, at what freshness, under which credential reference. Never a credential.

04

Evidence & provenance

Content-digested records with validity windows, traceable to every object and relationship that depends on them.

05

Guardrail policies

Per risk tier: minimum evidence strength, maximum evidence age, and whether a human must decide.

06

Declared purposes

With obligations, jurisdictions, and retention. A tier can be restricted to a subset. Undeclared purpose fails closed.

07

Assurance tests

Deterministic gates plus a golden-question suite that fails when a question stops routing where its author linked it.

08

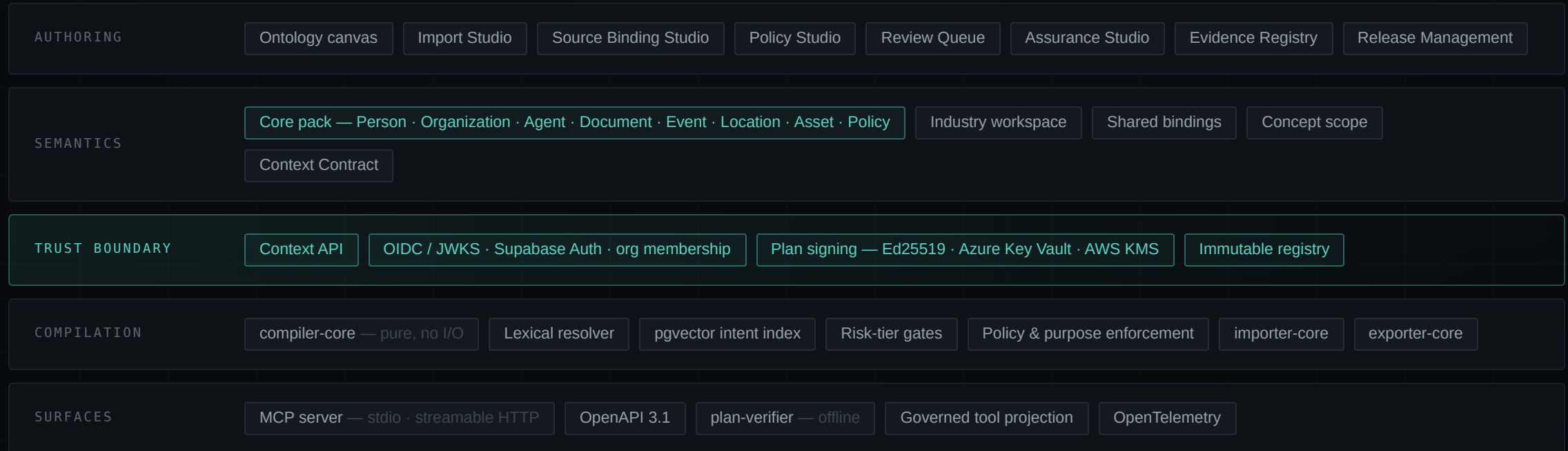
Release state

Immutable, SHA-256 addressed, diffable against any other release, restorable, and rollback-controlled with a mandatory rationale.

→ Publishing repeats every critical gate server-side. A UI cannot talk its way past a release check.

Five layers. One trust boundary.

Meaning is authored once at the top. Authority is held in exactly one place. Everything else is replaceable.



→ The compiler holds no key and opens no socket. Swap the resolver, the store, or the signer without touching what a decision means.

Seven gates. Every one of them can stop it.

And every stop produces a reason code, not an apology.

GATE 01 Identity Asymmetric JWT verified against your issuer. Organization membership resolved independently. The body cannot assert who it is.	GATE 02 Intent Lexical plus optional vector candidates. Risk-tier score, margin, and confirmation thresholds. Semantic-only picks must be confirmed.	GATE 03 Entities Governed objects resolved through the pinned concept scope. Ambiguity becomes a typed clarification, not a coin flip.	GATE 04 Evidence Strength and freshness checked against the tier's policy. Weak or stale context abstains rather than answering.	GATE 05 Purpose The declared purpose must exist in the contract and be permitted at this tier. Undeclared fails closed.	GATE 06 Policy Permissions derived from verified token scopes; wildcards stripped. High-risk tiers escalate to a named human.	GATE 07 Signing Contract, semantic, policy, binding, API, and metric versions pinned. Grounding pinned. Subject bound. Short expiry.
--	--	--	--	---	---	--

Pinned into every plan

The resolver's own reasoning

Model and index digest, candidate scores, the thresholds applied, the winning margin, matched governed questions, and whether a human confirmed the operation.

Pinned into every plan

Grounding: live or simulated

A plan resolved from documented sample payloads says so, inside the signature. Reference mode must be declared in the published release to be permitted at all.

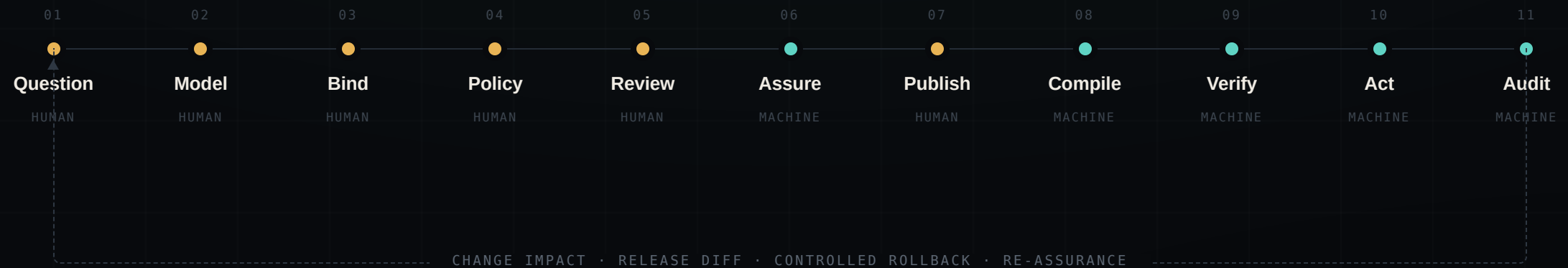
Enforced at execution

One subject, one use

Principal and tenant are signed in. A plan held by anyone else reports as absent, not forbidden. A rejected attempt does not burn the nonce.

One loop, run continuously — not a migration project.

Start with a single decision your team already makes by hand. The loop is the same at one contract and at five hundred.



● Human judgement ● Deterministic machinery

AI drafts. People approve consequential claims. The runtime enforces what was approved.

A thin plane between what you already run.

Lattice sits between your agent runtimes and your data platforms, and defers to your identity provider, your catalog, and your observability stack.

ABOVE — AGENT RUNTIMES CONSUME THE GOVERNED LOOP

Any MCP client

Claude Desktop

Bedrock AgentCore

Azure AI Foundry

Vertex Agent Builder

Databricks Agent Bricks

LangGraph

In-house orchestrators

6 MCP TOOLS · OPENAPI 3.1 · OFFLINE PLAN VERIFICATION · GOVERNED TOOL PROJECTION

LATTICE — GOVERNED CONTEXT PLANE

Compiles the question. Enforces the contract. Signs the plan. Records the receipt.
Owns meaning and authorisation — not storage, not identity, not the glossary.

Entra ID

Okta

Purview

Unity Catalog

Collibra

OpenTelemetry

DELEGATED IDENTITY · READ-ONLY SCOPES · SERVER-ONLY CREDENTIAL RESOLUTION · CLASSIFICATION FEDERATION

BELOW — GOVERNED SOURCE BINDINGS, READ-ONLY AND SCOPE-BOUNDED

Databricks NATIVE

Microsoft Fabric NATIVE

PostgreSQL NATIVE

Snowflake

BigQuery

Kafka

S3 · ADLS · OneLake

OpenAPI services

→ Credentials are resolved server-side from env:, vault, workload identity, or a broker — and never enter a contract, an export, a trace, or a browser response.

It does not want to be your *second* glossary.

A tool that competes with Purview, with Unity Catalog, or with the person who owns the semantic layer gets blocked in review. Three design decisions exist specifically to prevent that.

Your catalog stays authoritative

It does not re-decide what is sensitive.

Purview, Unity Catalog, and Colibra already own classification. Lattice imports their judgement at binding discovery and marks where it came from. A local judgement is only the fallback.

Detail on slide 14.

Your platform keeps enforcing

It does not stand in front of your access controls.

A binding can run the query as the person who asked, so Unity Catalog row filters and Fabric's own security apply because those systems apply them. The contract becomes an additional constraint, never a replacement.

Detail on slide 13.

Your semantic layer stays yours

It does not want to own your metrics.

Two definitions of “exposure” will disagree, and finance will side with the warehouse. The differentiating claim is compilation, evidence, and signed plans — none of which requires owning a metric registry.

Binding metrics by reference is still open work.

→ Every one of these is an adoption fight the product would lose. The next three slides are the proof, not the promise.

A binding is a governed claim, not a connection string.

It declares exactly what may be read, from where, at what age, keyed how, and as whom — and it goes through review like any other claim.

SOURCEBINDING	DATABRICKS · EXAMPLE
provider · transport	DATABRICKS over HTTPS — Statement Execution API 2.0
resource	warehouse · catalog · schema · object — every field required before it validates
queryTemplate	One read-only SELECT or WITH . Nothing else parses.
parameters	Which governed property supplies each argument — an LEI here, an internal account number there
parameterOrder	Required for positional providers, so adding an entity cannot silently rebind the query
maximumRows	50 by default, hard ceiling 1,000 a binding cannot raise
identityMode	SERVICE or DELEGATED — recorded on every receipt either way
credentialRef	env: · vault: · workload-identity: · managed-identity: — a reference, never a value
freshnessMinutes	The oldest this answer is allowed to be before the compiler abstains
requiredPermissions	Declared here, checked against the caller's verified token scopes
mappings	Source field → governed property, with type-compatibility review
sourceChecksum	What the schema looked like on the day it was approved
approvalStatus	Draft until a reviewer who is not the author decides
– not present –	No credential value. No connection secret. No token. Not in the contract, not in an export, not in a trace.

A read, not an extract

Bounded before it runs

Every query is wrapped and limited to **n+1** rows — enough for a decision, not enough for a bulk export, and one row past the ceiling so a receipt can say honestly whether it saw everything.

Read-only – and its honest limit

One statement, comments stripped, no side effects

Must begin **SELECT** or **WITH**. Comments are stripped before matching, so nothing hides behind them. Mutating keywords and the functions that read files or reach the network are refused.

A blocklist is not a SQL parser. Only PostgreSQL gets a real guarantee, from its **READ ONLY** transaction. The durable fix is a read-only role on your side.

Export safety

The sanitizer is mandatory

Portable JSON keeps approved credential references and strips sample payloads, secret-like fields, URL user information, and sensitive query parameters. RDF/XML and Turtle carry ontology meaning only.

Whether the query runs as *you* or as a service.

This is the difference between “governed” being true and being a claim. One shared credential means the row filters and column masks your team spent years building never see the person asking.

● IDENTITYMODE: SERVICE – THE DEFAULT

- 01 The person asks. Lattice verifies their token.
- 02 The binding resolves one shared service credential.
- 03 The platform sees the service principal.
- 04 Row filters and column masks apply to the service, not the user.

● IDENTITYMODE: DELEGATED

- 01 The person asks. Lattice verifies their token.
- 02 That token is exchanged — Entra on-behalf-of, or Okta RFC 8693.
- 03 The platform sees the person, scoped to that one resource.
- 04 Your controls apply. The contract is an additional constraint.

Per binding

Not a deployment-wide switch

A reference-data lookup can stay SERVICE while a customer-data read is DELEGATED. The decision sits where the risk is.

It fails, it does not fall back

Silence would be the dangerous outcome

A delegated binding with no exchange configured errors. Quietly running as the service principal would apply none of your controls while the receipt claimed the user's entitlements had.

Scoped per platform

A token for one is useless against the other

Databricks and Fabric each have their own resource identifier. A delegated token is no more broadly usable than the query that needs it.

Still open

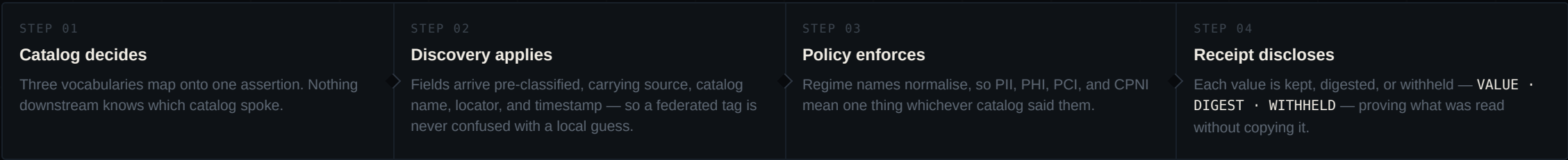
What we have not proven yet

Verification against a live Entra or Okta tenant, and delegation for Snowflake and BigQuery — each needs its own resource scopes.

Your catalog already decided. We just have to listen.

Asking an author to classify a column a second time in the Studio produces a diverging answer and a governance argument Lattice would lose. So classification is imported at discovery — the exact moment that second judgement would otherwise form.

CATALOG	WHAT CARRIES SENSITIVITY	WHAT CARRIES REGIME MEMBERSHIP
Microsoft Purview	The sensitivity label applied to the column	Classification rules Purview matched against its contents
Databricks Unity Catalog	A reserved tag key — sensitivity by default, configurable	Every remaining column tag
Collibra	Asset characteristics	Asset characteristics



Conservative by construction

An unrecognised label becomes **CONFIDENTIAL**, not **INTERNAL** — a catalog bothered to label the column, so treating an unfamiliar label as ordinary is the wrong way to be wrong. An unclassified property defaults to **INTERNAL**, never **PUBLIC**.

Degrades honestly

A catalog that cannot be read leaves fields unlabelled rather than blocking discovery — and is never reported as “nothing sensitive here”. Reads are bounded by a timeout and refuse redirects.

Still open

Dataplex, Alation, and Atlan adapters. Glossary term import with sameAs mappings. SKOS and DCAT export back to the catalog. OpenLineage events, so your lineage tools show which tables an agent decision consumed. Ownership resolved to a directory principal so approvals can route.

Six things that are very hard to retrofit.

Each of these had to be designed into the type system and the storage layer. None of them can be added later as middleware.

01

Separation of duties

The principal who submits a review cannot decide it. The person who requests a runtime approval cannot grant it, and cannot resume their own plan. Enforced in the store, so no interface can route around it.

02

Hash-chained ledgers

Assurance runs and execution receipts chain to their predecessor, and the store refuses to load when the chain does not verify. Verification recomputes each digest from content, so an edited field with a stale digest is caught.

03

Purpose limitation

Contracts declare purposes with obligations, jurisdictions, and retention. A policy can restrict a risk tier to a subset. The compiler fails closed on an undeclared or unpermitted purpose, and pins the declared one into the plan and the receipt.

04

Classification-aware receipts

Governed values above a threshold are withheld from the audit artifact rather than copied into it — enforced at the one place data would otherwise leak into permanent storage. Receipts record how many values were withheld.

05

Grounding pinned into the signature

A plan resolved from documented samples rather than live reads carries `grounding: SIMULATED` inside the signed payload. The MCP surface warns on it. Reference mode must be declared in the published release.

06

Release control

Immutable digest-backed releases, a digest-backed diff artifact between any two, restore-as-draft that never moves the live pointer, runtime suspension, and rollback that appends an actor-attributed event with a mandatory rationale.

→ Risk-tiered intent gates that pin their own scores, thresholds, and margin into the plan are a primitive almost nothing else ships.

Nine ontologies, derived — not hand-waved.

Generated deterministically from 74 implemented forms and 1,420 source fields. Every entity and property records the forms that contributed it; unmapped fields stay in the coverage report rather than disappearing from the audit trail.

Concepts start as DRAFT and TEMPLATE_DERIVED — human governance is required before publication.

INDUSTRY PACK		ENTITIES	RELATIONSHIPS	FORMS	FIELDS	COVERAGE
Telecommunications / NVO	PUBLISHED	19	24	11	196	100%
Airline	PUBLISHED	15	19	8	181	100%
Financial services	PUBLISHED	15	14	18	431	77.0%
Energy	PUBLISHED	4	3	4	70	74.3%
Healthcare		7	6	4	99	86.9%
Real estate		8	8	8	113	84.1%
Insurance		7	6	8	122	82.8%
Legal		6	5	5	104	76.9%
Manufacturing		8	7	8	104	75.0%
Total		89	92	74	1,420	84.6%

Published decision contracts

Airline

Dispatch release · return-to-service · passenger protection.

Published decision contracts

Telecommunications

Number porting · outage and 911 reporting · CPNI protection under 47 CFR 64.2005.

Published, end to end

Energy & financial services

Grid outage response traverses a governed relationship and returns a signed plan. Counterparty exposure exercises clarification and abstention.

→ No bank adopts a generated pack wholesale. The path that matters is the merge – import FIB0, HL7 FHIR, ACORD, or ISO 20022, align it to Core, and keep both provenance chains.

Customer proprietary network information.

47 CFR § 64.2005

A rule that distinguishes three uses of the same data. A contract that encodes exactly that distinction, and a compiler that fails closed on the third.

Step 01 – the request

An agent asks in the customer's own words and names a purpose. It cannot invent one — the permitted purposes are discoverable through the MCP surface.

Step 02 – purpose check

The contract declares the three uses the rule distinguishes and permits only two at its operational tier. The third is refused before any source is touched.

Step 03 – evidence & freshness

Policy requires **STRONG** evidence within a declared maximum age. Stale context abstains rather than answering with a caveat.

Step 04 – human authority

An **OPERATIONAL_ACTION** escalates. A different named principal decides, with a rationale. The agent cannot approve itself.

Step 05 – the plan

Signed. Pinned to the contract, semantic, policy, binding, API, and metric versions. Bound to one principal and tenant. Short expiry. Single use.

Step 06 – execution

Permissions derive from the executor's verified token scopes, wildcards stripped. A body that asserts its own entitlements is rejected outright.

Step 07 – the receipt

Chained to its predecessor. Every mapped value records whether it was kept, digested, or withheld — and why.

What a regulator gets

One digest-addressed artifact naming the purpose, the policy version, the evidence, the approver, and the exact release it compiled against.

Eight surfaces. One person can hold the whole loop.

The Studio is a working environment, not a diagram tool. Everything below writes to a governed draft that a release gate later has to accept.

Model

Ontology canvas

Drag types, draw relationships between node handles, edit properties in the inspector. Lane layout and semantic zones instead of a force-directed hairball.

Ingest

Import Studio

OpenAPI, JSON Schema, RDF/XML, Turtle, CSV. Every collision resolved as merge, separate, or skip. Checksum-stamped, and a proposal until you apply it.

Bind

Source bindings

Live metadata discovery on Databricks, Fabric, and PostgreSQL. Suggested property mappings, declared freshness, read-only scope, credential reference.

Constrain

Policy Studio

One guardrail profile per risk tier. Recommended baselines for uncovered tiers. Editing resets to draft and requires a fresh approval.

Approve

Review Queue

Types, bindings, and policies enter review as claims. Approve, approve-with-exception, or reject — each with a mandatory rationale, each immutable.

Prove

Assurance Studio

Structural, question, mapping, policy, and release gates, plus golden-question routing. Digest-backed artifacts. Critical failures block publish.

Trace

Evidence Registry

Provenance as a dependency graph. Answers “what becomes untrustworthy if this changes?” from the contract itself.

Ship

Release Management

Immutable history, downloadable diffs, semantic-version suggestions, runtime suspension, restore-as-draft, and rationale-backed rollback.

→ WCAG AA contrast, a 12px type floor with a large-text option, English and Spanish catalogues, and a pseudo-locale for finding untranslated strings — shipped, not planned.

Zero glue. The agent never gets free text back.

Every tool returns a governed outcome the runtime can branch on, plus structured content alongside the readable summary.

MCP TOOL	WHAT IT DOES	READ-ONLY
<code>lattice_list_contracts</code>	Contracts this identity may compile against. Start here.	yes
<code>lattice_describe_operations</code>	Operations as tool definitions, with tier, permissions, approval.	yes
<code>lattice_compile_question</code>	The main entry point. Returns one of the four outcomes.	no
<code>lattice_resolve_clarification</code>	Continues a paused resolution with a governed choice.	no
<code>lattice_verify_plan</code>	Signature, expiry, key, contract digest.	yes
<code>lattice_execute_plan</code>	Executes and returns a digest-backed receipt. Single use.	no

No tool accepts a token, and none ever will — a credential that can be passed in through model context is a credential the model can leak. The server acts as one governed service identity, configured from the environment; give each agent its own service principal if you want to tell them apart in the audit trail.

The compile path emits OpenTelemetry spans carrying decision and reason codes, risk tier, grounding, purpose, identity mode, and withheld-value count — so governed decisions land in your existing trace backend. Questions, source values, and tokens are never span attributes, and a test asserts it.

Self-describing

OpenAPI 3.1 at `/openapi.json`

Generate a client instead of reading a README. A test fails the build if a route is added without appearing in the document.

Tool projection

Contracts become tool lists

`projectGovernedTools` emits each operation as an Anthropic- or OpenAI-shaped definition carrying its risk tier, permissions, and approval requirement. The description states plainly that selecting it compiles rather than executes.

Independent verification

@`lattice/plan-verifier`

Signature, expiry, subject, tenant, and release pin — checked from a JWKS with no call back. Retired keys stay published through a rotation, and every applicable failure is returned at once.

Where this actually is, in August 2026.

The engineering team keeps an unsparing self-review in the repository — `enterprise-gaps.md` — naming every finding, its severity, and its status. This slide is that document, summarised.

● PROVEN AND VERIFIED

- The full loop — compile, clarify, approve, sign, verify, execute — run end to end across four distinct identities
- Permissions derived from verified token scopes, wildcards stripped, body-asserted entitlements rejected
- Plans bound to one subject and tenant; a rejected attempt does not consume the nonce
- KMS signing via Azure Key Vault and AWS KMS, RFC 7638 thumbprint key IDs, offline verification from a JWKS
- Separation of duties on reviews and approvals; hash-chained assurance and receipt ledgers
- Purpose limitation as policy input, plan pin, and audit field
- Classification federated from Purview, Unity Catalog, and Collibra into classification-aware receipts
- Delegated identity via Entra on-behalf-of and Okta RFC 8693 exchange
- MCP server, OpenAPI 3.1 with a drift test, governed tool projection, OpenTelemetry spans, full CI

● HARDENING NOW

- KMS adapters are implemented but not yet exercised against a live Azure or AWS account
- Delegated identity is covered by tests, not yet by a production Entra or Okta tenant
- Delegation for Snowflake and BigQuery — each needs its own resource scopes
- Measured freshness read from the source system, replacing authoring-time assertions
- Answer-level assurance assertions and binding probes; routing drift is already gated
- Tamper *evidence* is in. Resistance needs an external anchor — a signed head, a WORM store, or a witness
- Retention and an encryption boundary on receipt storage; rate limits on the compile path

● DELIBERATELY DEFERRED

- Environment promotion, contracts-as-code in Git, cross-contract impact analysis, deprecation lifecycle
- Self-hosted and air-gapped deployment profile
- Semantic-layer metric binding — dbt, Databricks metric views, Snowflake semantic views — rather than a parallel registry
- Glossary term import, SKOS and DCAT export, OpenLineage emission
- Standards merge path for FIBO, HL7 FHIR, ACORD, eTOM, ISO 20022
- Recorded agent-to-agent delegation

→ A vendor that cannot tell you what does not work yet is telling you something else instead.

Twelve weeks. One decision. One defensible artifact.

Not a platform rollout. Pick a single governed decision your team already makes by hand and put the whole loop around it.

PHASE 01 · WEEKS 1–3

Model one decision

Author the contract against your own vocabulary. Import an existing schema or standard rather than starting blank. Competency questions first; the ontology follows from them. Ends with a published release and a passing assurance run against fixtures.

PHASE 02 · WEEKS 4–8

Bind it to live systems

Delegated identity against your platform, so your row filters and column masks apply. Classification pulled from your catalog at discovery. Policy profiles reviewed and approved by the people who would have to defend them.

PHASE 03 · WEEKS 9–12

Put an agent on it

MCP into whichever runtime you already standardised on. Your executor verifies signed plans offline. Compile traces land in your observability stack next to the rest of the agent trace.

What we need from you

Four things, and a reviewer who cares

One governed decision with real consequences. One data platform. One identity provider. One person with the authority to approve a policy — because a governance product with no governor proves nothing.

What you keep

A contract you own, in formats we do not control

Portable JSON with governed bindings. OWL as RDF/XML and Turtle with stable IRIs. A digest-addressed release history. And a defensible answer to the question every agent program eventually gets asked.

Compilation before execution. *Signed plans* before tool calls.

- 01 Contracts before graphs.
- 02 Evidence before assertion.
- 03 Compilation before execution.
- 04 Signed plans before tool calls.
- 05 Clarification and abstention are first-class successful outcomes.
- 06 One core runtime; many independently versioned industry packs.